Day – 15 Neural Networks

Introduction To Neural Networks

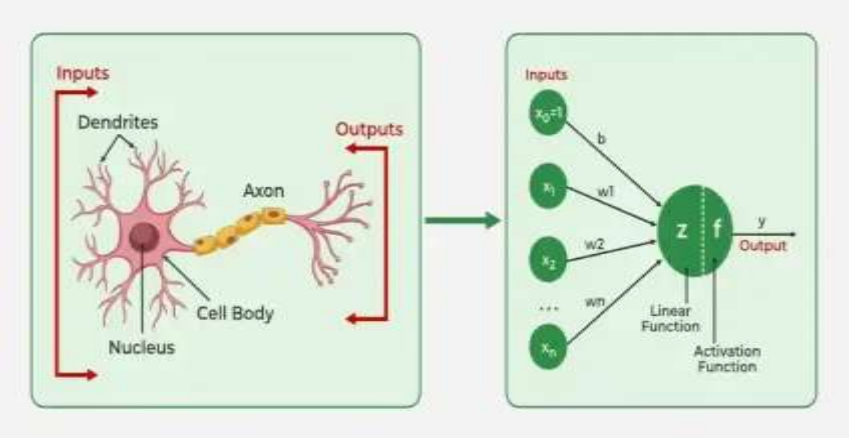
Neural networks are machine learning models that mimic the complex functions of the human brain. These models consist of interconnected nodes or neurons that process data, learn patterns directly from data without relying on pre-defined rules, enabling them to perform tasks such as pattern recognition and decision-making.

Neural Networks are built from several key components:

- **Neurons**: The basic units that receive inputs, each neuron is governed by a threshold and an activation function.
- **Connections**: Links between neurons that carry information, regulated by weights and biases.
- **Weights and Biases**: These parameters determine the strength and influence of connections.
- **Propagation Functions**: Mechanisms that help process and transfer data across layers of neurons.
- **Learning Rule**: The method that adjusts weights and biases over time to improve accuracy.

Learning in neural networks follows a structured, three-stage process:

1. **Input Computation**: Data is fed into the network.
2. **Output Generation**: Based on the current parameters, the network generates an output.
3. **Iterative Refinement**: The network refines its output by adjusting weights and biases, gradually improving its performance on diverse tasks.



Importance of Neural Networks

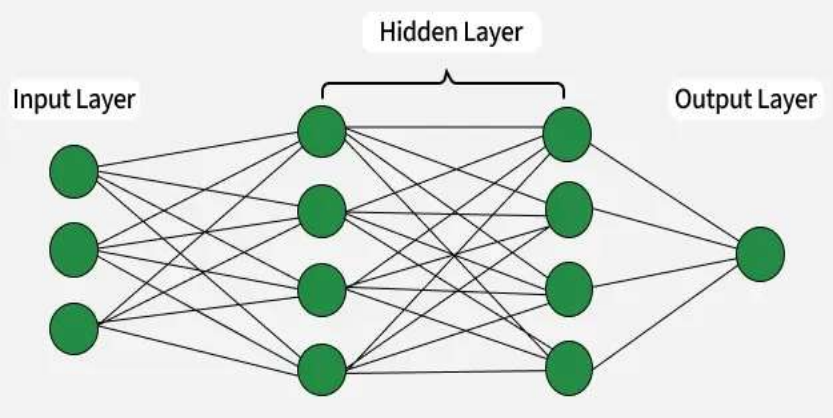
- **Identify Complex Patterns:** Recognize intricate structures and relationships in data; adapt to dynamic and changing environments.

- **Learn from Data:** Handle vast datasets efficiently; improve performance with experience and retraining.
- **Drive Key Technologies:** Power natural language processing (NLP); enable self-driving vehicles; support automated decision-making systems.
- **Boost Efficiency:** Streamline workflows and processes; enhance productivity across industries.
- **Backbone of AI:** Serve as the core driver of artificial intelligence progress; continue shaping the future of technology and innovation.

Layers in Neural Network Architecture

Layers in Neural Network Architecture

1. **Input Layer:** This is where the network receives its input data. Each input neuron in the layer corresponds to a feature in the input data.
2. **Hidden Layers:** These layers perform most of the computational heavy lifting. A neural network can have one or multiple hidden layers. Each layer consists of units (neurons) that transform the inputs into something that the output layer can use.
3. **Output Layer:** The final layer produces the output of the model. The format of these outputs varies depending on the specific task like classification, regression.



Working of Neural Networks

1. Forward Propagation

When data is input into the network, it passes through the network in the forward direction, from the input layer through the hidden layers to the output layer. This process is known as forward propagation. Here's what happens during this phase:

1. Linear Transformation: Each neuron in a layer receives inputs which are multiplied by the weights associated with the connections. These products are summed together and a bias is added to the sum. This can be represented mathematically as:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

where

- w represents the weights
- x represents the inputs
- b is the bias

2. Activation: The result of the linear transformation (denoted as z) is then passed through an activation function. The activation function is crucial because it introduces non-linearity into the system, enabling the network to learn more complex patterns. Popular activation functions include ReLU, sigmoid and tanh.

2. Backpropagation

After forward propagation, the network evaluates its performance using a loss function which measures the difference between the actual output and the predicted output. The goal of training is to minimize this loss. This is where backpropagation comes into play:

- **Loss Calculation:** The network calculates the loss which provides a measure of error in the predictions. The loss function could vary; common choices are mean squared error for regression tasks or cross-entropy loss for classification.
- **Gradient Calculation:** The network computes the gradients of the loss function with respect to each weight and bias in the network. This involves applying the chain rule of calculus to find out how much each part of the output error can be attributed to each weight and bias.
- **Weight Update:** Once the gradients are calculated, the weights and biases are updated using an optimization algorithm like stochastic gradient descent (SGD). The weights are adjusted in the opposite direction of the gradient to minimize the loss. The size of the step taken in each update is determined by the learning rate.

3. Iteration

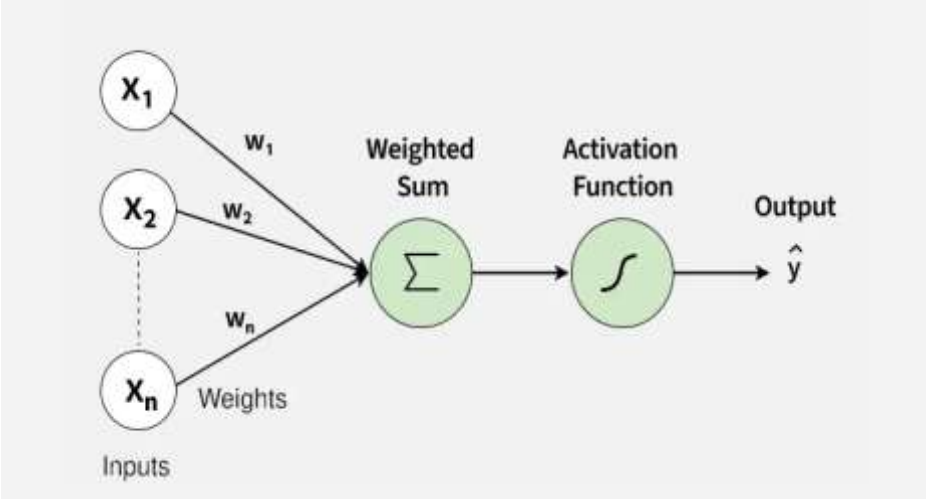
This process of forward propagation, loss calculation, backpropagation and weight update is repeated for many iterations over the dataset. Over time, this iterative process reduces the loss and the network's predictions become more accurate.

What is Perceptron?

A Perceptron is the simplest form of a neural network that makes decisions by combining inputs with weights and applying an activation function. It is mainly used for binary classification problems. It forms the basic building block of many deep learning models.

- Takes multiple inputs and assigns weights

- Computes a weighted sum and applies a threshold
- Outputs either 0 or 1 (binary outcome)
- Forms the foundation of larger neural networks



Perceptron vs. Multi-Layer Perceptron (MLP)

Aspect	Perceptron	Multi-Layer Perceptron (MLP)
Model Depth	Single layer with no hidden neurons.	Multiple layers with one or more hidden layers.
Type of Patterns Learned	Learns only linear relationships; straight-line separation.	Learns complex, non-linear patterns and curved boundaries.
Problem-Solving Ability	Cannot solve XOR or non-linearly separable problems.	Easily solves XOR and other complex classification tasks.
Activation Functions	Uses a simple step function (hard 0/1 output).	Uses advanced activations like ReLU, Sigmoid, Tanh for richer learning.
Learning Method	Trained with a simple perceptron update rule.	Trained using backpropagation and gradient descent.

Aspect	Perceptron	Multi-Layer Perceptron (MLP)
Real-World Use	Limited to simple demonstrations and basic classification.	Used in real-world AI systems like vision, NLP and deep learning applications.

Applications

- **Binary Classification**: Used for simple yes/no decision problems such as spam detection or basic quality checks.
- **Logic Gate Modelling**: Implements linearly separable gates like AND or NAND with perfect accuracy.
- **Pattern Recognition Basics**: Helps identify simple patterns where classes can be separated with a straight line.
- **Feature Importance Insight**: Weight values indicate which features influence the output more strongly.
- **Educational Tool**: Commonly used to teach foundations of machine learning and neural networks.

Advantages

- **Simple Architecture**: Easy to understand, implement and visualize as a basic neural model.
- **Fast Training**: Lightweight computations make learning very quick even on small devices.
- **Works on Linearly Separable Data**: Achieves perfect performance when a straight-line boundary exists.

Limitations

- **No Non-Linear Learning**: Fails on problems like XOR where a curved or complex boundary is needed.
- **Binary Output Only**: Cannot handle multi-class or probabilistic outcomes without extensions.
- **No Hidden Layers**: Lacks depth, making it unable to learn hierarchical or abstract patterns.